
Whilst we are waiting...

Go to <https://supabase.com> and create an account - this is where you will be creating your postgres database

Valencia Codes

Back to Basics:

Speaking the Language of Data: SQL

Data is power. SQL is the key.

Thanks to GFT

And Jose Miguel! :)



SQL in 30 seconds

- **1970:** Edgar F. Codd (IBM) proposes the **relational model** (tables, rows, set-based logic).
- **Early '70s:** IBM prototypes **SEQUEL** (later **SQL**) for System R.
- **Late '70s/'80s:** First commercial RDBMS (Oracle '79, DB2, Ingres).
- **Standardization:** **ANSI SQL-86/89**, major leap with **SQL-92**; later extensions (1999: recursive CTEs, 2003: XML/window funcs, 2011: temporal, 2016+: JSON/ROW_PATTERN in some engines).
- **Today:** SQL underpins Postgres, MySQL, SQL Server, Oracle, SQLite, plus data warehouses (Redshift, Snowflake, BigQuery). It's the lingua franca for data.

CRUD (the core 4) - Data Manipulation Language - DML

- **Create** → **INSERT** (and schema via DDL: **CREATE TABLE**)
- **Read** → **SELECT** (with **WHERE**, **GROUP BY**, **ORDER BY**, joins)
- **Update** → **UPDATE ... SET ... WHERE ...**
- **Delete** → **DELETE FROM ... WHERE ...**

Nice-to-know:

- **UPSERT:** **INSERT ... ON CONFLICT DO UPDATE** (Postgres) / **MERGE** (ANSI style).
- **Transactions:** **BEGIN; ... COMMIT;** (or **ROLLBACK**) for atomic, **ACID**-safe changes.
- **DDL vs DML:** DDL = define schema (**CREATE/ALTER/DROP**), DML = change data (**INSERT/UPDATE/DELETE/SELECT**).



Why SQL & Data Still Matter in the Age of AI

- Foundation of AI – SQL provides the structure AI models rely on. (no more mention of AI from this point on! 🤨)
- Data Quality = Trust – Ensures reliable insights and predictions
- Scalability & Business Value – Translates data into revenue & decisions
- Human Oversight – Lets humans interrogate numbers directly
- Universal Skillset – Critical across industries and tools.



Decisions

AI

Analytics

SQL

Data



Why a Database (Not Just Excel)?

- Scalability
 - Excel slows down with thousands of rows. Databases handle millions to billions efficiently.
- Data Integrity
 - Databases enforce rules (types, constraints, relationships). Excel relies on manual discipline → error-prone.
- Multi-User Access
 - Databases allow many people to query and update data at once. Excel struggles with collaboration and version control.
- Security & Governance
 - Fine-grained access controls and audit trails in databases; Excel has limited protection.
- Automation & Integration



Some common types of databases

- MSSQL (RDBMS)
- Postgres (RDBMS)
- MySQL (RDBMS)
- Mongo (NOSQL)
- Redshift (Columnar)
- Snowflake (Columnar)
- Big Query (Columnar)
- And many others (which are less important! 🤪)



SQL Best Practices for Beginners

1. Write Clear & Readable Queries

- Use **uppercase for SQL keywords** (`SELECT`, `WHERE`, `JOIN`) and lowercase for tables/columns → improves readability.
- **Indent and format** queries so logic is easy to follow.
- Add **comments** (`-- this filters active customers`) for clarity.

2. Always Select Explicitly

- Avoid `SELECT *` in production queries.
 - It's fine for exploration, but always **name columns** to:
 - Improve performance.
 - Prevent errors if schemas change.
 - Make results self-explanatory.

3. Filter Data Thoughtfully

- Use `WHERE` clauses early to avoid scanning unnecessary rows.
- Prefer `=` and `IN` where possible over `LIKE` if not searching text.
- Be mindful of **NULL values** (`IS NULL` vs `= NULL`).

4. Consistency Matters

- Decide on a **style guide** (snake_case vs camelCase, singular vs plural table names).
- Stick to it so queries are predictable.



SQL query order execution

```
SELECT ...  
FROM ...  
JOIN ... ON ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
WINDOW ...  
ORDER BY ...  
LIMIT/OFFSET
```



What actually happens...

1. **FROM** (base tables, subqueries, CTEs are bound)
2. **JOIN ... ON** (row matching; for outer joins, unmatched outer rows are kept)
3. **Add outer rows** (LEFT/RIGHT/FULL OUTER effect realized)
4. **WHERE** (filters **rows**; window/aggregate not allowed here)
5. **GROUP BY** (rows collapse into groups)
6. **HAVING** (filters **groups**)
7. **WINDOW** / analytic functions (computed over the post-group result)
8. **SELECT** (expressions evaluated; aliases created)
9. **DISTINCT** (deduplicate selected rows)
10. **ORDER BY** (can refer to aliases or column positions)
11. **LIMIT/OFFSET** (top-N / pagination)



But how do we connect and start writing SQL?

IDE's (Integrated Development Environment) - that's how!

- DBeaver
- SSMS
- Toad
- DataGrip

And many more...

Let's go!

A light-colored dog is looking at a baby who is sitting in a white wicker chair. The dog is on the left, and the baby is on the right. The background is a mottled grey.

**AN SQL QUERY GOES INTO A BAR,
WALKS UP TO TWO TABLES AND ASKS...**

A light-colored dog is looking at a baby who is sitting in a white wicker chair. The dog is on the left, and the baby is on the right. The background is a mottled grey.

'CAN I JOIN YOU?'